COMP20005 Workshop Week 6

0	Questions on Last Week Ex: 5.1-5.3, 5.5, 5.6?
1	Scopes, *exercise 6.2*
2	*Recursive Functions (time permits)*
3	Pointers, Pointers as Function Arguments, *discuss: exercise 6.5, discuss exercise 6.9*
L **A** **B**	*implement 6.9 with a pointer argument* *further improve 6.9 with an array* *other exercises from chapters 6 and 5*

Assignment 1: available on the LMS from Friday 17 April,

- **discussed in the lecture Week 7 on Monday 20 April,**
- **Q&A during the workshop Week 7,**
- **due 6PM Friday Week 8 (May 1).**

Scopes: local variables

```
#include <stdio.h>
int fact(int n);
int main(int argc, char *argv[]){
   int n= 3;
   int factorial= fact(n);
   printf("%d! = %d\n", n, factorial);
   return 0;
}
int fact(int k) {
   int i, factorial= 1;
   for (i=1; i<=k; i++) {
      factorial *= i;
   }
   return factorial;
}
```

argc, **argv**, **n**, and **factorial** only available here

k, **i**, and **factorial** only available here

All function variables and parameters are local.

Scopes: global variables (and functions)

Global declarations (not inside any function)

```
#include <stdio.h>
int world;
int foo(int n);

int main(int argc, char *argv[]){
    int n= 3, val;
    world= 100;
    val= foo(n);
    printf("val= %d, world= %d\n", val, world);
    return 0;
}

int foo(int n) {
    return n+world;
}
```

scope of functions **foo**

scope of global variable **world**

***6.02**:*
For each of the 3 marked points, write down a list of all of the program-declared variables and functions that are in scope at that point, and for each identifier, its type.

```
1   int bill(int jack, int jane);
2   double jane(double dick, int fred, double dave);
3
4   int trev;
5
6   int main(int argc, char *argv[]) {
7       double beth;
8       int pete, bill;     /* -- point #1 -- */
9       return 0;
10  }

11  int bill (int jack, int jane) {
12      int mary;
13      double zack;        /* -- point #2 -- */
14      return 0;
15  }

16  double jane(double dick, int fred, double dave) {
17      double trev;        /* -- point #3 -- */
18      return 0.0;
19  }
```

Stuffs to remember

- For a function: its arguments and its declared variables are local and can be accessed only inside the function.
- In main(), if we call a function foo with foo(a,b), there is no way foo can change the value of a and b (which are local variables of main).
- Do not use global variables
- Do not use a same identifier for different purposes

Recursive Functions (time permits)

What: A function that calls itself.

When:

- a task of size n can be reduced to a a same task of a smaller size (like n-1), and
- solution for small, trivial case can be easily found (base case).

Example: compute f(n) = n! (mathematically)

$$= 1*2*3*\ldots*(n-1)*n$$

$$= (\ 1*2*3*\ldots*(n-1)\ )\ *\ n$$

$$= (n-1)!\ *\ n$$

2 different ways to compute, say, 3!

We arrive to a th recurrence:

- f(n) = f(n-1)*n if n>1 (general case) and
- f(1) = 1 (base case)

Recursive Functions (time permits)

What: A function that calls itself.

When:

- a task of size n can be reduced to a a same task of a smaller size (like n-1), and
- solution for small, trivial case can be easily found (base case).

Example: compute f(n) = n!

$$= 1*2*3*\ldots*(n-1)*n$$

$$= (\ 1*2*3*\ldots*(n-1)\ )\ \ *\ n$$

$$= (n-1)!\ \ *\ \ n$$

- f(n) = f(n-1)*n (general case) and
- f(1) = 1 (base case)

```
int factorial( int n ) {
  if (n==1) {
    return 1;
  }
  return n * factorial(n-1);
}
```

How a recursive call works?

```
int main ...
  ...

  k=   f(3);
  ...
}
```

```
int f(int n) { //n=3
  if (3<=1) return 1;

  return 3*f(2);
}
```

```
int f( int n ) {
  if (n<=1) {
    return 1;
  }
  return n*f(n-1);
}
```

```
int main ...
  ...

  k=   f(3);
  ...
}
```

```
int f(int n) { //n=3
  if (3<=1) return 1;

  return 3*f(2);
}
```

```
int f(int n) { //n=2
  if (2<=1) return 1;

  return 2*f(1);
}
```

```
int f( int n ) {
  if (n<=1) {
    return 1;
  }
  return n*f(n-1);
}
```

```
int main ...
  ...

  k=   f(3);
  ...
}
```

```
int f(int n) { //n=3
  if (3<=1) return 1;

  return 3*f(2);
}
```

```
int f(int n) { //n=2
  if (2<=1) return 1;

  return 2*f(1);
}
```

```
int f(int n) { //n=1
  if (1<=1) return 1;

  return n*f(n-1);
}
```

```
int f( int n ) {
  if (n<=1) {
    return 1;
  }
  return n*f(n-1);
}
```

```
int main ...
  ...

  k=   f(3);
  ...
}
```

```
int f(int n) {
  if (2<=1) return 1;

  return 3* f(2) ;
}
```

```
int f(int n) {
  if (2<=1) return 1;

  return 2* f(1) ;
}
```

```
int f(int n) {
  if (1<=1) return 1;

  return n*f(n-1);
}
```

1

```
int f( int n ) {
  if (n) {
    return 1;
  }
  return n*f(n-1);
}
```

```
int main ...
  ...

  k=   f(3);
  ...
}
```

```
int f(int n) {
  if (2<=1) return 1;

  return 3* f(2) ;
}
```

```
int f(int n) {
  if (2<=1) return 1;

  return 2* f(1) ;
}
```

1

2

```
int f( int n ) {
  if (n) {
    return 1;
  }
  return n*f(n-1);
}
```

```
int main ...
  ...

  k=   f(3);
  ...
}
```

```
int f(int n) {
  if (2<=1) return 1;

  return 3* f(2) ;
}
```

2

6

```
int f( int n ) {
  if (n) {
    return 1;
  }
  return n*f(n-1);
}
```

```
int main ...
  ...

  k=   f(3) ;   6
  // tadaa: k is 6
  ...
}
```

```
int f( int n ) {
  if (n) {
    return 1;
  }
  return n*f(n-1);
}
```

Recursive functions: How

- reduce the task of size n to the same tasks of smaller sizes
- clearly describe the base cases where the solutions are trivial
- when writing code, start with solving the base cases first

Examples:

factorial (n):

- base case: when n==1 the solution is 1
- general case: factorial(n) can be computed from factorial(n-1)

```
int factorial( int n ) {
  if (n==1) { // do the base case first!
    return 1;
  }
  // general case  [note: else is not needed here]
  return   factorial(n-1)*n;
}
```

Examples: 5.13

5.13 Write a *recursive* function int_pow that raises its first argument to the power of its second argument, where the first argument is an int, and the second argument is a positive int. For example, int_pow(3, 4) should return the value 81.

Other exercises on recursion: 5.14, 5.15

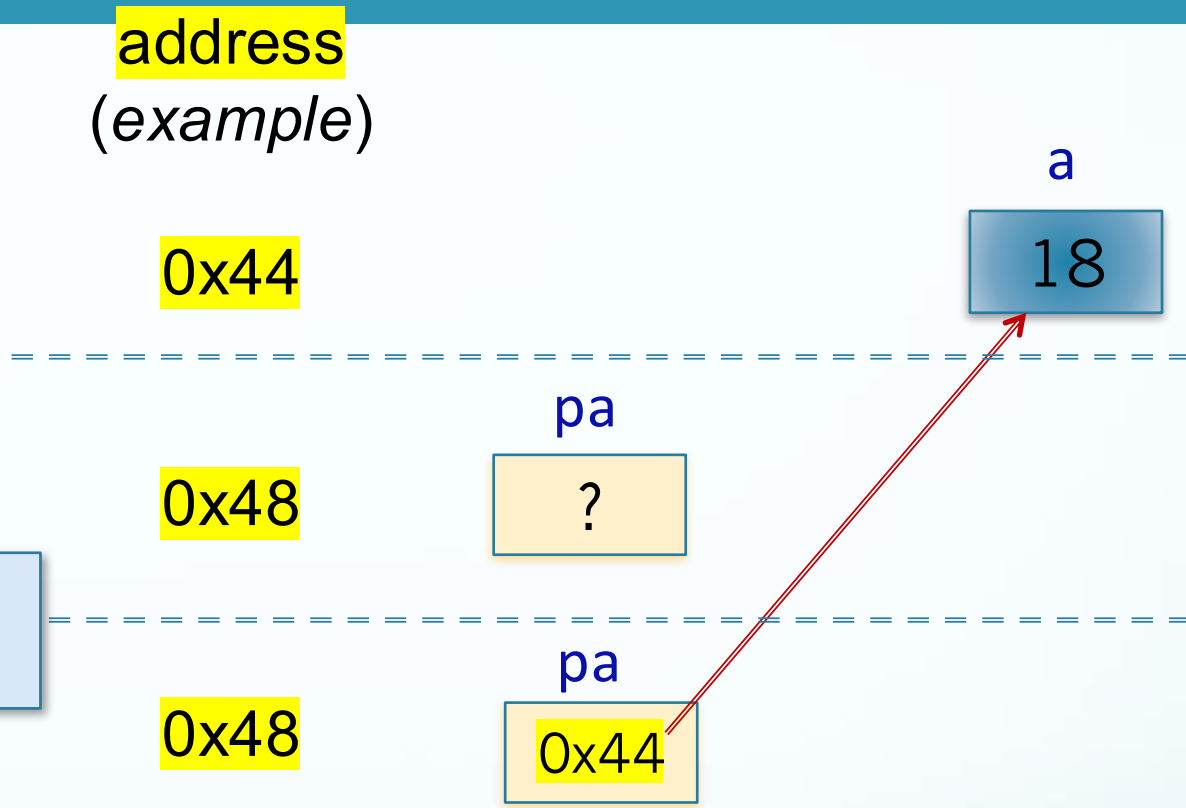
Pointers: check your understanding

```
int a= 18;

int *pa;

pa= &a;
```

here pa, not *pa, is declared!
here * is a part of declaration int *




```
// What is the value of a and pa after:
*pa = (*pa) + 1;
```

here *pa, not pa, is used and modified!
here * is a unary operator.

Pointers: check your understanding

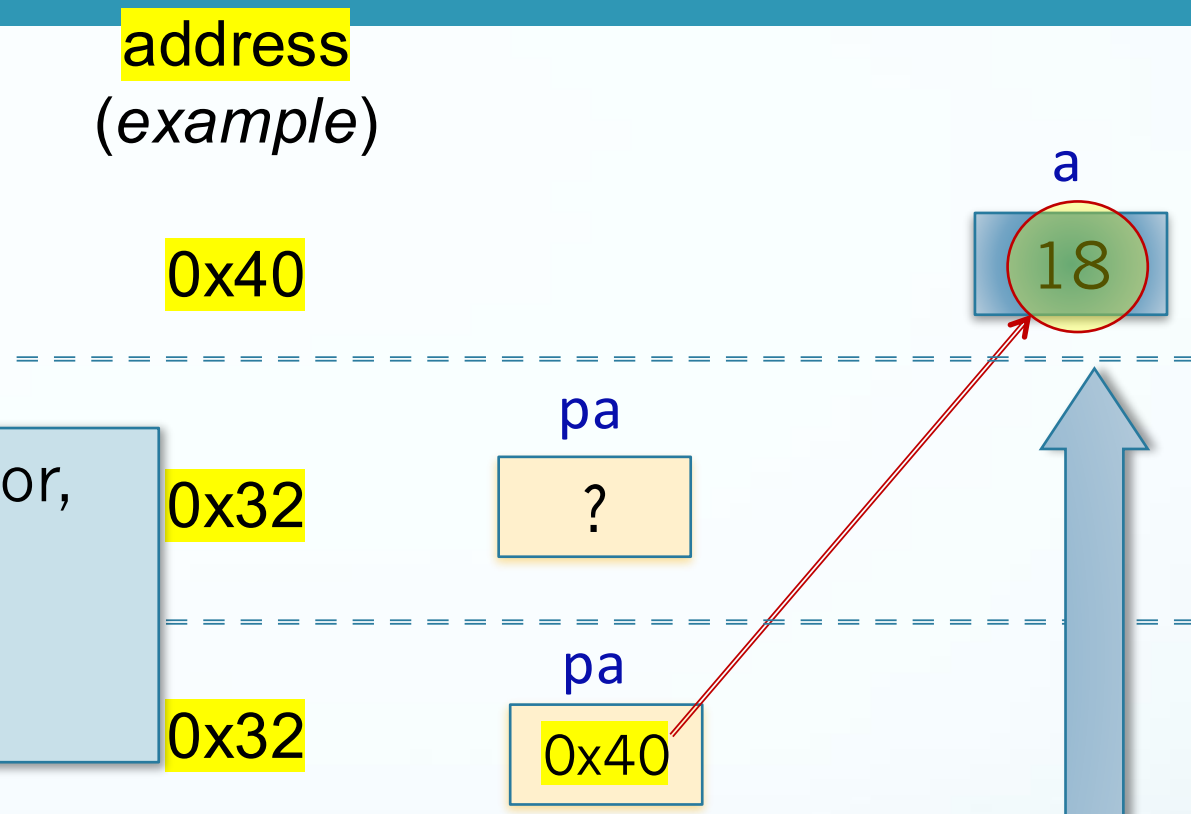
```
int a= 18;
int *pa;

pa= &a;
```

& (address-of) is a unary operator,
&a gives address of a (**0x40**)
here pointer **&a** points to
the memory location **0x40**

```
// What is the value of a and pa after:
*pa = (*pa) + 1;
```

== increase the value of **a** (same as ***pa**) by 1 to 19



* (dereference) is a unary operator
***X** gives the value at memory address **X**
***ptr** gives the value at the location **ptr** points to

unary operators & and * : referencing and dereferencing

```
int n= 10;
int *pn;

pn= &n;
```

Check your understanding:

a) The datatype of pn is _____

b) If n is at the address 0x4444, then pn has the value of _____

c) The value of *pn is _____

d) After

```
*pn= 100;
```

the value of pn is _____, of n is _____

e) What is the effect of:

```
*(&n) = 1;
```

Pointers – application in function parameters

1	int n=10;	
2	printf("%d", n);	What sent to printf ? Can printf change the value of n?
3	scanf("%d", **&n**);	What sent to scanf? Why?
4	int_swap(&n, &m);	What passed to swap? Can this call make change to &n or &m? Can this call make change to m or n?

5	void int_swap(int *a, int *b){
	???
	}	

a function problem: working as intended? why?

```
1   int main(...) {
2     int a=2, b=4, sum= 0, product= 0;
3       ...
4     sAndP(a, b, sum, product);
5     printf("sum=%d",      sum);
6     printf("prod=%d",  product);
7       ...
8   }

    // computes sum= m+n and prod= m*n
11  void sAndP(int m, int n, int sum, int prod ) {

12    sum = m + n;
13    prod = m * n ;

14  }
```

a function problem: working as intended? why?

Q: How to make sAndP change the value of **sum** and **product**?

```
int main(...) {
  int a=2, b=4, sum= 0, product= 0;
     ...
  sum= sAndP(a, b, sum, product);
  printf("sum=%d",      sum);
  printf("prod=%d",  product);
    ...
}

// computes sum= m+n and prod= m*n
void sAndP(int m, int n, int sum, int prod ) {

   sum = m + n;
   prod = m * n ;

}
```

sum and **product** can never be changed with this function call

a solution: sending addresses instead of values

Line 4 leads to the change of **sum** and **product**.

```
1   int main(...) {
2     int a=2, b=4,                    sum, product;
3       ...
4     sAndP(a, b, &sum, &product);
5
6     printf("sum= %d", sum);
7     printf("product= %d", product);
8       ...
9   }

11  void sAndP(int m, int n, int *psum, int *pprod ) {

12    *psum = m + n ;
13
      *pprod = m * n ;
14
    }
```

Here, ***psum** and ***pprod** (with *) represent two outputs.
They are just **int**

Discuss, then do Exercise 6.05, then 6.09, then 6.08

6.05 In the context of this main(), write a function sort2
so that it re-orders the values of v1 and v2 in the increasing order.
For example:

- If v1, v2 are 10, 5 before calling sort2, they become 5, 10 after calling.
- If v1, v2 are 1, 2 before calling sort2, they remain 1, 2 after calling.

```
int main(int argc, char *argv[]) {
    ...

    printf("Before: v1 = %d, v2 = %d\n", v1, v2);
    sort2( ??? );
    printf("After:  v1 = %d, v2 = %d\n", v1, v2);

    return 0;
}

??? sort2( ??? ) {

    ...

}
```

Lab

- Implement 6.9 (and Re-implement 6.5 if still in doubt)
- MST: implement or write-solution-on-paper (or whiteboard) for not-yet-done Exercises in Chapter 05
- *Do the exercise with* triangle.c *as described in LMS Week 6 Workshop Content and other exercises from Chapter 6*

Assignment 1 released this Fri, discussed in Mon lecture!

- Do as much as you can by Week 7 Workshop
- Try to submit at least once
- Regularly use Discussion Forum
- Q&A in Week 7

Additional Slides

Quiz 1

In executing the program:

```
int a=100, b=200;
void f(int a) {
   a++;
   print("1: a= %d b= %d\n", a, b) ;
}
int main(int argc, char *argv[]) {
  int a=5, b= 10;
  f(a);
  print("2: a= %d b= %d\n", a, b) ;
  return 0;
}
```

what will be printed out?:

A 1: a= 6 b= 200 2: a= 5 b= 10	B 1: a= 6 b= 200 2: a= 6 b= 10
C 1: a= 6 b= 10 2: a= 5 b= 10	D 1: a= 6 b= 10 2: a= 6 b= 10

Quiz 2

After executing the fragment:

```
int x= 10;
f(&x);
printf("x= %d\n", x);
```

the output is:

```
x= 0
```

which function has been used in the call f(&x) *?*

A:

```
int f(int n) {
   return 0;
}
```

B:

```
void f( int *n) {
  &n= 0;
}
```

C:

```
void f (int *n) {
   n= 0;
}
```

D:

```
void f( int *n) {
  *n= 0;
}
```

Quiz 3

Given function:

```
void f(int a, int *b) {
  a= 1;
  *b = 2;
}
```

Assuming the following fragment is in a valid main(). What will be printed out?

```
int m= 5;
int n= 10;
f(m, &n);
printf("m= %d, n= %d\n", m, n);
```

A) m= 5, n= 10	B) m= 1, n= 2
C) m= 5, n= 2	D) m=1, n= 10

Check your answers: Quizzes

Q1: A

Function f() prints before main(). Inside f(), b refers to the global variable with a value of 200. Conversely, in main(), both a and b are local variables that retain their initial values regardless of any operations performed within f().

Q2: D

Option A has an incorrect input type. Options B and C are invalid as they both attempt to modify a memory address directly.

Q3: C

Option C is correct: $m = 5, n = 2$. The function call f(m, &n) can only modify n because it is passed by address (pointer), allowing the function to access its memory location. Conversely, m is passed by value, so the original variable remains 5. The value of n is updated to 2 by the final assignment within function f.